Computación gráfica y ambientes virtuales (0494)

Práctico 01

21 de agosto de 2026

Parte 1 — El triángulo básico dibujado usando OpenGL

Usando el template del proyecto de OpenGL disponible en el aula virtual, las Guías de aula A y B y las filminas de la clase práctica, dibujar un triángulo de color sólido.

Antes de cerrar la actividad, hacer el ejercicio de la clínica de las filminas: sacar un punto y coma al vertex shader y volver a correr. Si no aparece ningún mensaje en la consola, falta pedir el log.

Parte 2 — El módulo `ResourceManager` (C++ puro, sin OpenGL)

En el programa básico los dos shaders viven *adentro* del archivo `.cpp`, como strings. Para poder escalar el programa hay que procesarlos como archivos externos (ej. `*.vs` y `*.fs`) y para eso hace falta un módulo que los lea desde el disco, **que no los lea dos veces** y que mantenga una copia en memoria para ser utilizada por otras partes del proyecto.

Este módulo habrá que ampliarlo más adelante para administrar otros recursos del proyecto como texturas, modelos de mallas poligonales, etc.

Funcionalidades que debe tener.

- C++ puro: sin llamadas a la API de OpenGL, sólo biblioteca estándar.
- Lee de disco el fuente de un shader y lo deja disponible como texto.
- Si el mismo recurso se pide dos veces, **la segunda no vuelve a leer del disco**.
- Si algo falla — ej. no está el archivo, no está la carpeta, se pide algo que no se cargó — hay que generar un **aviso**: no se rompe en silencio.

Por dónde empezar. La interfaz propuesta se muestra a continuación considerando las funcionalidades para shaders unicamente. Es una recomendación, no un molde. La forma — un `struct` por tipo de recurso, un `load_*`, un `get_*` y un mapa para almacenar los recursos — permitirá ampliar más adelante el módulo para gestionar otro tipo de recursos (texturas, modelos, etc.).

```
// ResourceManager.h  --- punto de partida, no molde
#pragma once
#include <string>
#include <filesystem>
#include <unordered_map>

struct ShaderSource {           // codigo fuente de un programa de shader
    std::string vs;             // vertex shader
    std::string fs;             // fragment shader
    std::string gs;             // geometry shader (opcional, hoy no se usa)
};

class ResourceManager {
public:
    explicit ResourceManager(const std::filesystem::path& assets_root);

    const ShaderSource& load_shader_source(const std::string& key,
                                           const std::string& vs_file,
                                           const std::string& fs_file,
                                           const std::string& gs_file = "");
    const ShaderSource& get_shader_source(const std::string& key) const;
    void clear(void);

private:
    std::filesystem::path assets_root_;
    std::unordered_map<std::string, ShaderSource> shaders_sources_;

    std::string read_shader_file(const std::string& shader_file,
                                 const std::string& shader_type);
};
```

Algunas cuestiones a pensar.
Los puntos a continuación tiene más de una respuesta razonable, y para algunos, la interfaz propuesta arriba ya eligió una.

- ¿El cache se indexa por una **clave** que elige el usuario del módulo, o directamente por el nombre del archivo?
- Si se pide usar una clave que ya está cargada pero con archivos distintos: ¿se recarga, se ignora lo nuevo, es un error?
- ¿Cómo se avisa que algo falló: excepción, valor de retorno u otra cosa?
- ¿Devuelve una referencia `const` o una copia? ¿Qué pasa con esa referencia si alguien llama a `clear()` después?
- ¿Hace falta un `struct`, o alcanza con guardar los fuentes sueltos?

Algunas recomendaciones

1. Crear `main.cpp` de prueba que **demuestre que el cacheo funciona**.
2. Documentar mínimamente en la interfaz las decisiones tomadas.
3. Que compile así, sin warning:
   `g++ -std=c++17 -Wall -Wextra ResourceManager.cpp main.cpp -o test`
4. Usar sólo la biblioteca estándar de C++. No hace falta nada extra.